



## 프로젝트 확장

- 특정 감독(Director)별 평균 평점 분석
- 특정 배우(Star) 별 평균 평점 비교
- 특정 조건(예: 평점 8.0 이상 & 2000년 이후 제작) 충족하는 영화만 필터링

## 확장 prepare:

- 기존 Step1 데이터를 기반으로 **프로젝트 확장 분석용 컬럼** 포함
- **Director**, **Star** 컬럼을 추가로 선택하고, 기존 컬럼( **Title**, **Genre**, **Rate** )과 함께 NumPy 배열로 변환

### 처리 순서

1. CSV 파일 읽기 ( **IMDB Top 1000.csv** )
2. 큰 따옴표 안/밖 쉼표 구분 파싱 ( **parse\_csv\_line** )
3. 필요한 컬럼 선택:
  - 기존: **Title**, **Genre**, **Rate**
  - 확장: **Director**, **Star** 추가 선택
4. Title 전처리
  - 앞 숫자 제거
  - 괄호 안 연도 추출 ( **Released\_Year** )
5. 결측값 제거
6. 데이터 타입 변환
  - Rate → float
  - Released\_Year → int
7. NumPy 배열 변환 ( **dtype=object** )
8. 확장용 **.npy** 저장 ( **data\_array\_extension.npy** )

### 코드

```
import numpy as np

# csv 파일 경로
csv_path = "../01_data/IMDB top 1000.csv"

# csv 파싱 함수 정의
# 큰 따옴표 안에 있는 쉼표와 바깥에 있는 쉼표 구분
def parse_csv_line(line):
    row = []      # 한 줄을 분리한 문자열 담을 리스트
    current = ""   # 현재 문자열 누적
    inside_quotes = False # 큰 따옴표 안에 있는지 여부 체크
    for char in line:
        if char == '"' and not inside_quotes: # 큰 따옴표 시작
            inside_quotes = True
        elif char == '"' and inside_quotes: # 큰 따옴표 종료
            inside_quotes = False
        elif char == ',' and not inside_quotes: # 따옴표 밖에 쉼표
            row.append(current) # 여태 누적된 문자열 리스트 안에 담기
            current = ""      # 누적 초기화
        else:
            current += char    # 문자 누적
    row.append(current)      # 마지막 문자열 리스트 안에 담기
    return row              # for문 밖에서 반환

# csv 파일 읽기
with open(csv_path,"r", encoding="utf-8") as f:
    lines = f.read().splitlines() # 줄 단위 리스트 생성

# 헤더와 데이터 분리 후
header = parse_csv_line(lines[0]) # 파싱 헤더 추출
data_lines = lines[1:]          # 데이터 줄만 따로 저장
```

```

# -----
# 기존 Step1에서 선택된 컬럼 + 확장용 컬럼 추가
# -----
title_idx = header.index("Title") # 1
genre_idx = header.index("Genre") # 4
rate_idx = header.index("Rate") # 5
cast_idx = header.index("Cast") # 8 → 확장 Step1 : 감독/배우 정보 파싱용

# 데이터 전처리 리스트 준비
processed_data = [] # 최종 NumPy 배열로 변환할 데이터 담을 리스트

# 데이터 줄 반복 처리
for line in data_lines:
    row = parse_csv_line(line) # 파싱하여 컬럼 분리

    # row 길이가 필요한 컬럼 중 가장 큰 인덱스보다 작으면 해당 컬럼이 없다는 의미
    if len(row) <= max(title_idx, genre_idx, rate_idx):
        continue # 없다면, 건너 뛴다. 다음 줄을 처리하러 for문 처음으로 돌아감.

    # 컬럼 값 가져오기 및 공백 제거
    Num_title = row[title_idx].strip()
    genre = row[genre_idx].strip()
    rate = row[rate_idx].strip()

    # 확장 step1: Cast 컬럼에서 Direction/Stars 추출
    cast_info = row[cast_idx].strip()
    director = []
    stars = []
    if "Director:" in cast_info and "|" Stars:" in cast_info:
        # Director 추출
        director_str = cast_info.split("Director:")[1].split("| Stars:")[0].strip()
        # Stars 추출
        stars_str = cast_info.split("| Stars:")[1].strip()

        # ',' 기준으로 리스트 분리 + 공백 제거
        directors = [d.strip() for d in director_str.split(",") if d.strip()]
        stars = [s.strip() for s in stars_str.split(",") if s.strip()]

    # Title 앞에 있는 숫자 제거
    title = Num_title.split('.',1)[1].strip() if '.' in Num_title else Num_title

    # 결측값 제거 : 빈 문자열이 있으면 해당 행 제외
    if "" in [title, genre, rate] or len(directors) == 0 or len(stars) == 0:
        continue

    # Title에서 Released_Year 추출
    # Title 마지막에 () 형태가 있어야만 슬라이싱
    released_year = None
    if '(' in title and title[-1] == ')':
        released_year = title[title.rfind('(')+1 : title.rfind(')')]
    else:
        # 괄호가 없으면 임의로 None 처리
        released_year = None

    # 타입 변환 후 리스트에 추가
    try:
        processed_data.append([title, genre, float(rate), int(released_year), directors, stars])
    except ValueError:
        # 변환 불가능한 값이 있으면 건너뛰기
        continue

# NumPy 배열로 변환
# dtype=object 사용 : 문자열 + 숫자가 혼합된 배열
data_array = np.array(processed_data, dtype=object)

# 결과 확인
print("NumPy 배열 형태의 데이터 출력: ", data_array)
print("\n데이터 크기(shape) 확인: ", data_array.shape)

# NumPy 배열 바이너리 파일로 저장

```

```
np.save("../03_outputs/results/data_array_extension.npy", data_array)
```

## 결과

배열 형태의 데이터 출력: `[['The Shawshank Redemption (1994)' 'Drama' 9.3 1994 list(['Frank Darabont']) list(['Tim Robbins', 'Morgan Freeman', 'Bob Gunton', 'William Sadler'])]`  
`[['The Godfather (1972)' 'Crime, Drama' 9.2 1972 list(['Francis Ford Coppola']) list(['Marlon Brando', 'Al Pacino', 'James Caan', 'Diane Keaton'])]`  
`[['The Dark Knight (2008)' 'Action, Crime, Drama' 9.0 2008 list(['Christopher Nolan']) list(['Christian Bale', 'Heath Ledger', 'Aaron Eckhart', 'Michael Caine'])]`  
...  
`[['JFK (1991)' 'Drama, History, Thriller' 8.0 1991 list(['Oliver Stone']) list(['Kevin Costner', 'Gary Oldman', 'Jack Lemmon', 'Walter Matthau'])]`  
`[['Nights of Cabiria (1957)' 'Drama' 8.1 1957 list(['Federico Fellini']) list(['Giulietta Masina', 'François Périer', 'Franca Marzi', 'Dorian Gray'])]`  
`[['Throne of Blood (1957)' 'Drama, History' 8.1 1957 list(['Akira Kurosawa']) list(['Toshirô Mifune', 'Minoru Chiaki', 'Isuzu Yamada', 'Takashi Shimura'])]`

데이터 크기(shape) 확인: `(887, 6)`

```
Numpy 배열 형태의 데이터 출력: [['The Shawshank Redemption (1994)' 'Drama' 9.3 1994 list(['Frank Darabont']) list(['Tim Robbins', 'Morgan Freeman', 'Bob Gunton', 'William Sadler'])]
['The Godfather (1972)' 'Crime, Drama' 9.2 1972 list(['Francis Ford Coppola']) list(['Marlon Brando', 'Al Pacino', 'James Caan', 'Diane Keaton'])]
['The Dark Knight (2008)' 'Action, Crime, Drama' 9.0 2008 list(['Christopher Nolan']) list(['Christian Bale', 'Heath Ledger', 'Aaron Eckhart', 'Michael Caine'])]
...
['JFK (1991)' 'Drama, History, Thriller' 8.0 1991 list(['Oliver Stone']) list(['Kevin Costner', 'Gary Oldman', 'Jack Lemmon', 'Walter Matthau'])]
['Nights of Cabiria (1957)' 'Drama' 8.1 1957 list(['Federico Fellini']) list(['Giulietta Masina', 'François Périer', 'Franca Marzi', 'Dorian Gray'])]
['Throne of Blood (1957)' 'Drama, History' 8.1 1957 list(['Akira Kurosawa']) list(['Toshirô Mifune', 'Minoru Chiaki', 'Isuzu Yamada', 'Takashi Shimura'])]

데이터 크기(shape) 확인: (887, 6)

종료 코드 0(으)로 완료된 프로세스
```

### 기존 Step1과 다른 점 / 확장점

| 항목               | 기존 Step1           | 확장 Step1                            |
|------------------|--------------------|-------------------------------------|
| 컬럼 선택            | Title, Genre, Rate | Title, Genre, Rate + Director, Star |
| Released_Year 처리 | 기존과 동일             | 주석 명시, int 변환                       |
| 결측값 제거           | 기존 컬럼만             | 확장 컬럼 포함                            |
| 저장 파일            | data_array.npy     | data_array_extension.npy            |
| 활용               | Step2~Step5 분석     | 감독/배우/조건 필터링 분석 가능                  |

리스트 컴프리헨션에서 `if d.strip()` / `if s.strip()` 은 공백만 있는 항목은 무시하고, 실제 이름만 리스트에 넣는다.

## 확장 Step1: 특정 감독별 평균 평점

 [파이썬 한글 시각화용 코드 설명](#)

 [사용 코드 설명](#)

```
import matplotlib.pyplot as plt
import matplotlib.font_manager as fm
import numpy as np

# =====
```

```

# 파이참 한글 시각화 설정
# =====
font_path = "C:/Windows/Fonts/malgun.ttf"
fontprop = fm.FontProperties(fname=font_path, size=12)
plt.rcParams['axes.unicode_minus'] = False

# =====
# Director - 감독별 평균 평점 분석
# =====
data_array = np.load("../03_outputs/results/data_array_extension.npy", allow_pickle=True)

# 감독별 평점 수집
director_ratings = {}
for row in data_array:
    rate = row[2]
    for director in row[4]:
        director_ratings.setdefault(director, []).append(rate)

# 평균 평점 계산
director_avg = {d: round(sum(r)/len(r),2) for d,r in director_ratings.items()}

# 내림차순 정렬
director_sorted = sorted(director_avg.items(), key=lambda x: x[1], reverse=True)

# =====
# 출력: 상위 10명
# =====
top10 = director_sorted[:10]
print("감독별 평균 평점 상위 10명:")
for name, score in top10:
    print(f"{name} - 평균 평점: {score}")

# =====
# 시각화 준비
# =====
# 이름과 점수 리스트를 역순으로 (그래프용)
names = [item[0] for item in top10][::-1]
scores = [item[1] for item in top10][::-1]

# =====
# Bar Plot
# =====
plt.figure(figsize=(10, 6))

plt.barh(names, scores, color='lightblue', edgecolor='black')
for x, y in zip(scores, names):
    plt.text(x + 0.02, y, str(x), va='center', fontproperties=fontprop, fontsize=10)

plt.xlabel("평균 평점", fontproperties=fontprop)
plt.ylabel("감독 이름", fontproperties=fontprop)
plt.title("감독별 평균 평점 상위 10명 (Bar Plot)", fontproperties=fontprop)
plt.grid(axis='x', linestyle='--', alpha=0.3)
plt.tight_layout()
plt.show()

# =====
# Dot Plot
# =====
plt.figure(figsize=(10, 6))

plt.scatter(scores, names, s=120, color='skyblue')
for x, y in zip(scores, names):
    plt.text(x + 0.02, y, str(x), va='center', fontproperties=fontprop, fontsize=10)

plt.xlabel("평균 평점", fontproperties=fontprop)
plt.ylabel("감독 이름", fontproperties=fontprop)
plt.title("감독별 평균 평점 상위 10명 (Dot Plot)", fontproperties=fontprop)
plt.grid(axis='x', linestyle='--', alpha=0.3)
plt.tight_layout()
plt.show()

```

결과

감독별 평균 평점 상위 10명:

Frank Darabont - 평균 평점: 8.95

Francis Ford Coppola - 평균 평점: 8.87

Peter Jackson - 평균 평점: 8.8

Thomas Kail - 평균 평점: 8.7

Irvin Kershner - 평균 평점: 8.7

Robert Zemeckis - 평균 평점: 8.65

Roberto Benigni - 평균 평점: 8.6

Jonathan Demme - 평균 평점: 8.6

George Lucas - 평균 평점: 8.6

Masaki Kobayashi - 평균 평점: 8.6

감독별 평균 평점 상위 10명:

Frank Darabont - 평균 평점: 8.95

Francis Ford Coppola - 평균 평점: 8.87

Peter Jackson - 평균 평점: 8.8

Thomas Kail - 평균 평점: 8.7

Irvin Kershner - 평균 평점: 8.7

Robert Zemeckis - 평균 평점: 8.65

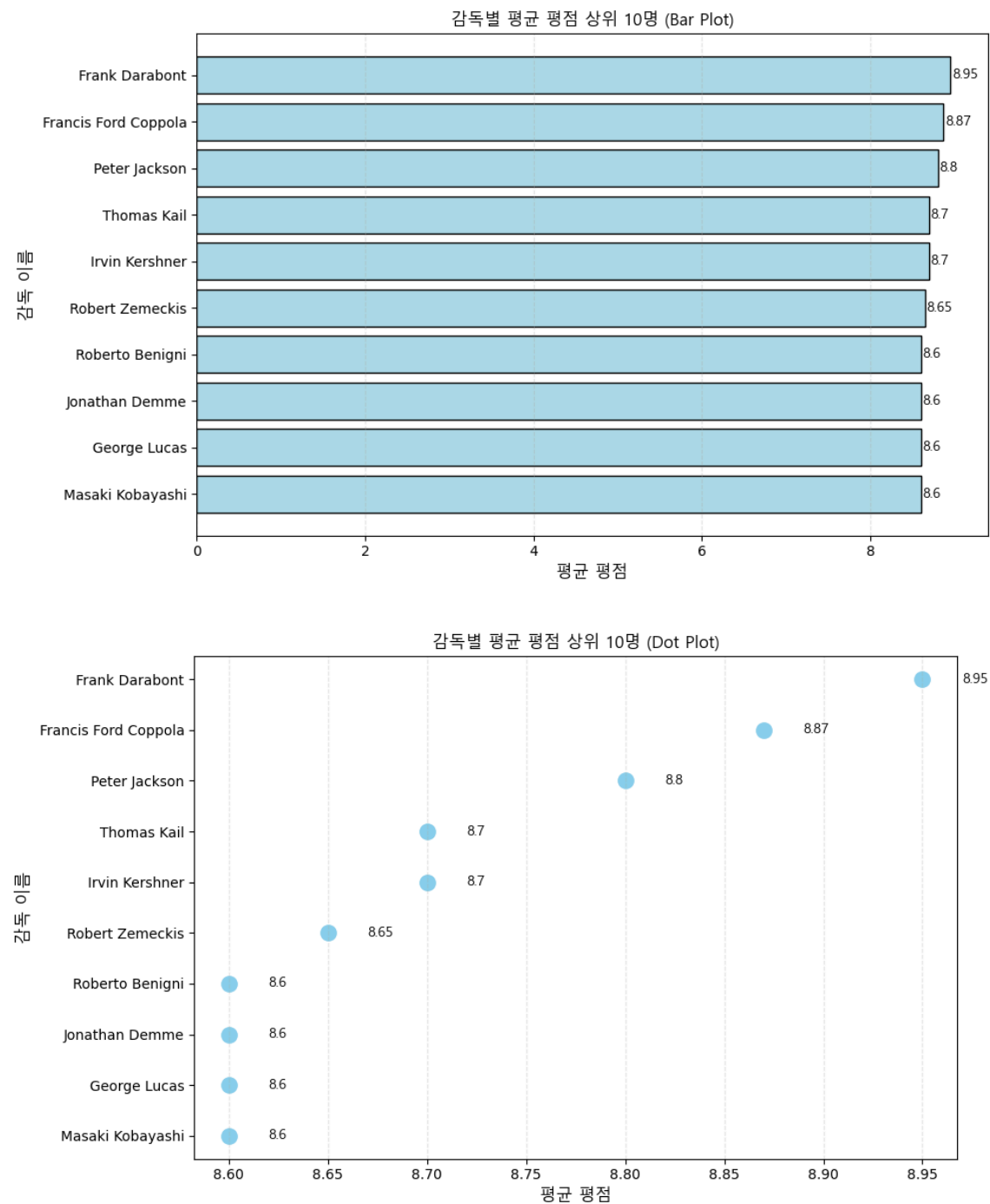
Roberto Benigni - 평균 평점: 8.6

Jonathan Demme - 평균 평점: 8.6

George Lucas - 평균 평점: 8.6

Masaki Kobayashi - 평균 평점: 8.6

종료 코드 0(으)로 완료된 프로세스



확장 Step2: 특정 배우별 평균 평점

 사용 코드 설명

```
import matplotlib.pyplot as plt
import matplotlib.font_manager as fm
import numpy as np

# =====
# 파이참 한글 시각화 설정
# =====
font_path = "C:/Windows/Fonts/malgun.ttf" # 맑은 고딕
fontprop = fm.FontProperties(fname=font_path, size=12) # 폰트 속성 객체 생성
plt.rcParams['axes.unicode_minus'] = False # 음수 표시 문제 방지

# =====
# star - 배우별 평균 평점 분석
```

```
# =====

# 데이터 로드
data_array = np.load("../03_outputs/results/data_array_extension.npy", allow_pickle=True)

# 배우별 평점 수집
star_ratings = {}
for row in data_array:
    rate = row[2]
    for star in row[5]:
        star_ratings.setdefault(star, []).append(rate)

# 배우별 평균 평점 계산
star_avg = {s: round(sum(r)/len(r),2) for s,r in star_ratings.items()}

# 평균 평점 기준 내림차순 정렬
star_sorted = sorted(star_avg.items(), key=lambda x: x[1], reverse=True)

# =====
# 출력: 상위 10명
# =====
top10_stars = star_sorted[:10]
print("배우별 평균 평점 상위 10명:")
for name, score in top10_stars:
    print(f"{name} - 평균 평점: {score}")

# =====
# 시각화 준비
# =====
# 이름과 점수 리스트를 역순으로 (그래프용)
names = [s[0] for s in top10_stars[::-1]]
scores = [s[1] for s in top10_stars[::-1]]

# =====
# 1) Bar Plot
# =====
plt.figure(figsize=(10,6))
plt.barh(names, scores, color='salmon', edgecolor='black')
for x, y in zip(scores, names):
    plt.text(x + 0.02, y, str(x), va='center', fontproperties=fontprop, fontsize=10)
plt.xlabel("평균 평점", fontproperties=fontprop)
plt.ylabel("배우 이름", fontproperties=fontprop)
plt.title("배우별 평균 평점 상위 10명 (Bar Plot)", fontproperties=fontprop)
plt.grid(axis='x', linestyle='--', alpha=0.3)
plt.tight_layout()
plt.show()

# =====
# 2) Dot Plot
# =====
plt.figure(figsize=(10,6))
plt.scatter(scores, names, s=120, color='salmon')
for x, y in zip(scores, names):
    plt.text(x + 0.02, y, str(x), va='center', fontproperties=fontprop, fontsize=10)

plt.xlabel("평균 평점", fontproperties=fontprop)
plt.ylabel("배우 이름", fontproperties=fontprop)
plt.title("배우별 평균 평점 상위 10명 (Dot Plot)", fontproperties=fontprop)
plt.grid(axis='x', linestyle='--', alpha=0.3)
plt.tight_layout()
plt.show()
```

## 결과



배우별 평균 평점 상위 10명:

Tim Robbins - 평균 평점: 9.3

Bob Gunton - 평균 평점: 9.3

William Sadler - 평균 평점: 9.3

Diane Keaton - 평균 평점: 9.1

Heath Ledger - 평균 평점: 9.0

Aaron Eckhart - 평균 평점: 9.0

John Travolta - 평균 평점: 8.9

Caroline Goodall - 평균 평점: 8.9

Martin Balsam - 평균 평점: 8.9

John Fiedler - 평균 평점: 8.9

배우별 평균 평점 상위 10명:

Tim Robbins - 평균 평점: 9.3

Bob Gunton - 평균 평점: 9.3

William Sadler - 평균 평점: 9.3

Diane Keaton - 평균 평점: 9.1

Heath Ledger - 평균 평점: 9.0

Aaron Eckhart - 평균 평점: 9.0

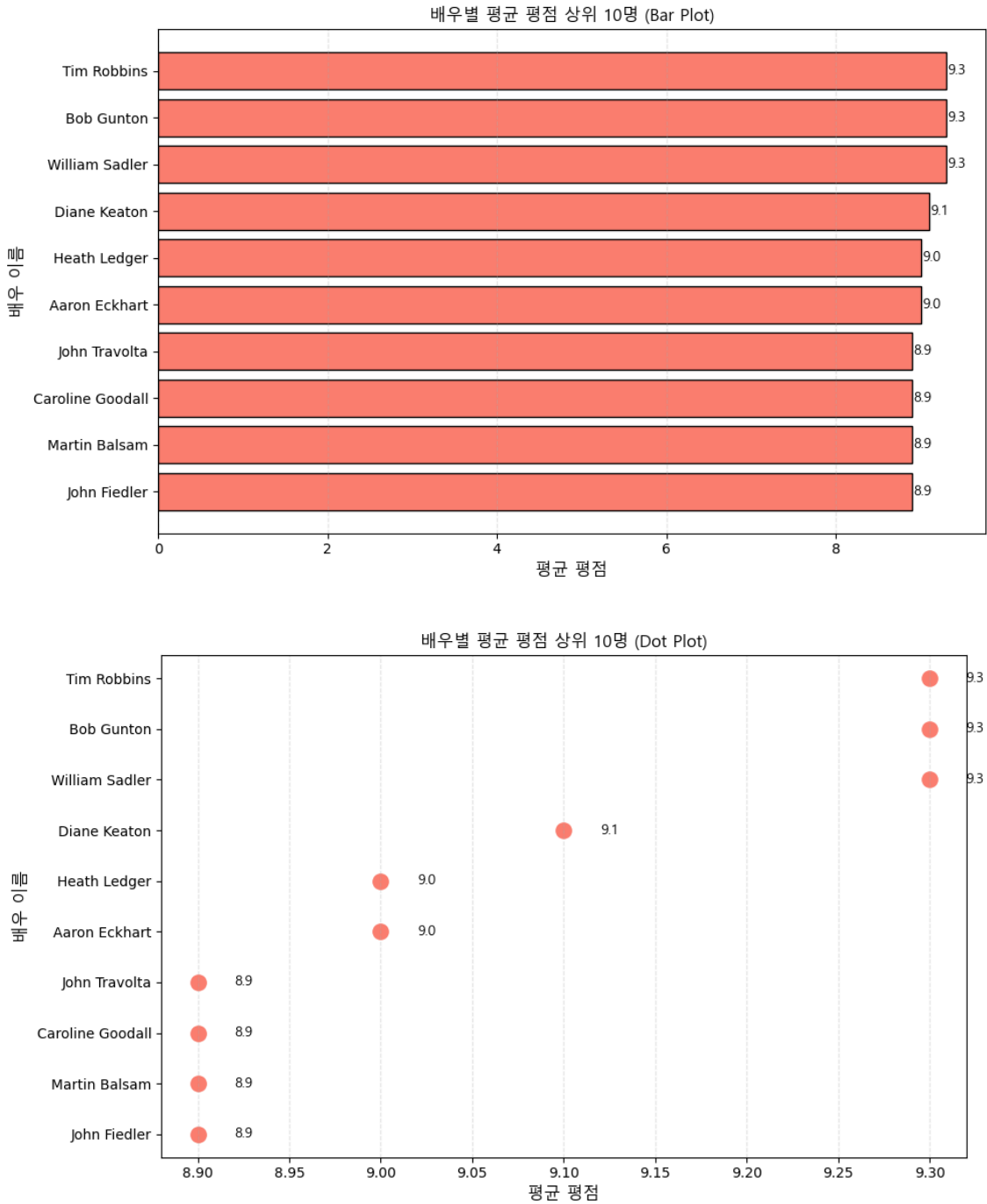
John Travolta - 평균 평점: 8.9

Caroline Goodall - 평균 평점: 8.9

Martin Balsam - 평균 평점: 8.9

John Fiedler - 평균 평점: 8.9

종료 코드 0(으)로 완료된 프로세스



## 확장 Step3: 특정 조건 충족하는 영화만 필터링

[🔗 사용 코드 설명](#)

```
import matplotlib.pyplot as plt
import matplotlib.font_manager as fm
import numpy as np

# =====
# 파이참 한글 시각화 설정
# =====
font_path = "C:/Windows/Fonts/malgun.ttf" # 한글 폰트 경로 (맑은 고딕)
fontprop = fm.FontProperties(fname=font_path, size=12)
plt.rcParams['axes.unicode_minus'] = False # 음수 표시 문제 방지

# =====
# 데이터 로드 및 필터링 (평점 8.0 이상, 2000년 이후 제작 영화)
# =====
```



```

data_array = np.load("../03_outputs/results/data_array_extension.npy", allow_pickle=True)

min_rate = 8.0
min_year = 2000

filtered = []
for row in data_array:
    title, rate, year = row[0], row[2], row[3]
    if rate >= min_rate and year >= min_year:
        filtered.append([title, rate, year])

filtered_array = np.array(filtered, dtype=object)

# =====
# 출력: 총 영화 수 및 상위 10개 영화 정보
# =====
print(f"총 영화 수 (평점 {min_rate} 이상 & {min_year}년 이후 제작): {len(filtered_array)}\n")
print("상위 10개 영화 (평점 높은 순):")
# 평점 기준 내림차순 정렬
top10_movies = sorted(filtered_array, key=lambda x: x[1], reverse=True)[:10]
for movie in top10_movies:
    print(f"{movie[0]} - 평점: {movie[1]}")

# =====
# Dot Plot & 히스토그램을 위한 데이터 준비
# =====
rates = filtered_array[:,1].astype(float)
years = filtered_array[:,2].astype(int)

unique_years = np.unique(years)
avg_rates = []
movie_counts = []

for y in unique_years:
    y_rates = rates[years == y]
    avg_rates.append(np.mean(y_rates))
    movie_counts.append(len(y_rates))

sizes = [c*20 for c in movie_counts] # 점 크기 조정

# =====
# 시각화
# =====
fig, axes = plt.subplots(1, 2, figsize=(14,5)) # 서브플롯 2개

# 1) 히스토그램 - 평점 분포
axes[0].hist(rates, bins=np.arange(8, 10.1, 0.2), color='lightblue', edgecolor='black')
axes[0].set_xlabel("평점", fontproperties=fontprop)
axes[0].set_ylabel("영화 수", fontproperties=fontprop)
axes[0].set_title("평점 분포 (8.0 이상)", fontproperties=fontprop)

# 2) 연도별 평균 평점 & 영화 수 Dot Plot
axes[1].scatter(unique_years, avg_rates, s=sizes, color='salmon', alpha=0.6, edgecolor='black')
for x, y, c in zip(unique_years, avg_rates, movie_counts):
    axes[1].text(x, y + 0.02, str(c), ha='center', va='bottom', fontproperties=fontprop, fontsize=9)
axes[1].set_xlabel("제작 연도", fontproperties=fontprop)
axes[1].set_ylabel("평균 평점", fontproperties=fontprop)
axes[1].set_title("연도별 평균 평점 & 영화 수", fontproperties=fontprop)
axes[1].grid(True, linestyle='--', alpha=0.3)

plt.tight_layout()
plt.show()

```

## 결과

총 영화 수 (평점 8.0 이상 & 2000년 이후 제작): 496

상위 10개 영화 (평점 높은 순):

The Dark Knight (2008) - 평점: 9.0

The Lord of the Rings: The Return of the King (2003) - 평점: 8.9

Inception (2010) - 평점: 8.8



The Lord of the Rings: The Fellowship of the Ring (2001) - 평점: 8.8  
 Hamilton (2020) - 평점: 8.7  
 The Lord of the Rings: The Two Towers (2002) - 평점: 8.7  
 Parasite (2019) - 평점: 8.6  
 Interstellar (2014) - 평점: 8.6  
 Spirited Away (2001) - 평점: 8.6  
 Joker (2019) - 평점: 8.5

